Data Description

Samatrix Consulting Pvt Ltd

Samatrix.io

Descriptive Statistics

- We can divide the field of statistics into two major branches:
  - descriptive statistics
  - inferential statistics
- For both the branches, we need a set of measurements to work upon.
- If the objective of the study is a data description, we generally consider the data of the entire population.
  - For example, the objective is to study the distribution of annual family income for the whole population in India as per the year 2011 census, we do not require a random sample because the data is already available.
  - We can use the available data of the whole population and focus on organizing, summarizing, and describing the data.
- The descriptive statistics help us understand the data by summarizing the large dataset to a few statistical measures that provide a picture of the original dataset.

Inferential Statistics

- If we are interested in statistical inference, we use sampling methods.
- From the distribution of numbers in the sample, we can gain meaningful insights about the distribution of numbers for the whole population.
- Still, for the statistical inference studies, we need to organize, summarize, and describe the sample data.

Examples:

- Hypothesis Testing:
  - **T-tests**: Compare means between groups.
  - **Chi-square tests**: Test for independence between variables.
- Confidence Intervals:
  - Range in which a population parameter likely lies.
- Regression Analysis:
  - Determines relationships between variables.
- Analysis of Variance (ANOVA):
  - Compares means across multiple groups.

Data Visualization

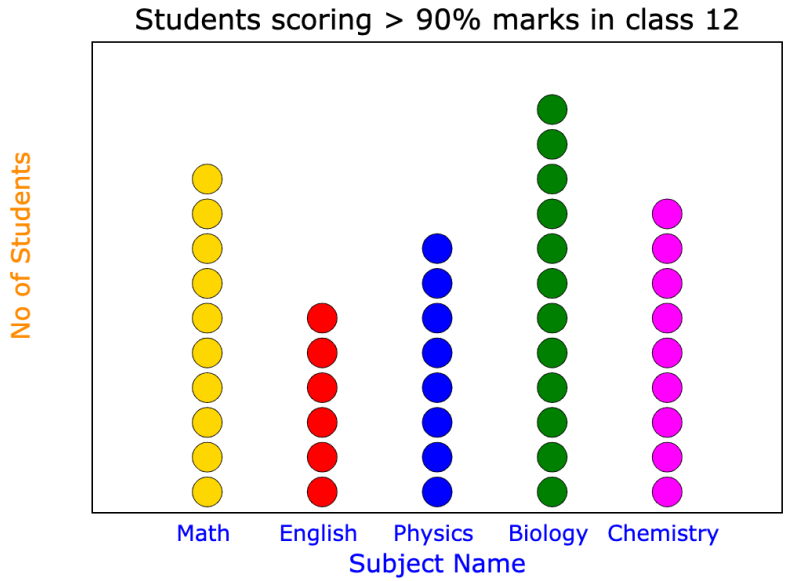
- We get a limited understanding of data by looking at the set of numbers.
- We need ways to present the data in such a way, that allows us to note the significant features of the data.
- Our brain cannot process the data in row format very well but helps us perceive the data in a pictorial format.
- But the correct picture of data is required.
- An incorrect picture can lead us to wrong interpretations.

Graphically displaying a single variable

Visual Display of Measurement

- On several occasions, our data set contains a set of measurements on a single variable for a single sample.
- Using this, we want to gain insights into the distribution of measurement of the population.
- We can use the visual display of measurements of the sample.

Dot Plots



- The simplest way to display data for a single variable is a dot plot.
- In this graphical representation, each value is represented by a dot along the horizontal axis.
- It also shows the relative positions of all the observations that also helps to get a general idea of the distribution of the values.
- Example: 45 students scored more than 90% marks in various subjects for class 12 board examinations. The dot plot distribution is as follows

Samatrix.io

Box Plots -1

- Boxplot is another simple graphical method that can be used to summarize the distribution of the data.
- The first step is to sort and summarize the data.
- In the raw data set, the sample values are $y_1, y_2, \cdots, y_n$. Where the subscript denotes the sample number.
- We get the **order statistics** by ordering the sample values from smallest to largest.
- After ordering, we can denote them as $y_{[1]}, y_{[2]}, \cdots, y_{[n]}$ where $y_{[1]}$ is the smallest, $y_{[2]}$ is second smallest and $y_{[n]}$ is the largest value.
- Now, we divide the ordered observations into the quartiles, Q_1, Q_2, Q_3.

Box Plots-2

- Q_1 is the lower quartile. The value of Q_1 is 25% of the values that are less than or equal to it and 75% of the values that are greater than or equal to it.
- Q_2 also known as sample median, is the middle quartile. The value of Q_2 is 50% of the values that are less than or equal to it and 50% of the values that are greater than or equal to it.
- Q_3 is the upper quartile. The value of Q_3 is 75% of the values that are less than or equal to it and 25% of the values that are greater than or equal to it.

$$Q_1 = y_{\left[\frac{n+1}{4}\right]}$$

$$Q_2 = y_{\left[\frac{n+1}{2}\right]}$$

$$Q_3 = y_{\left[\frac{3(n+1)}{4}\right]}$$

- If the result is not an integer, the averages of two closest values of the ordered data set are taken.

Box Plots-3

- For example, if our sample contains 29 data sets which gives $n = 29$

$$Q_1 = y_{\left[\frac{30}{4}\right]}$$

- We need to determine the value of 7th and 8th data of the ordered dataset and take the average. So

$$Q_1 = \frac{1}{2} \times y_{[7]} + \frac{1}{2} \times y_{[8]}$$

- We now get a five-number summary of a data set $y_{[1]}, Q_1, Q_2, Q_3, y_{[n]}$.
- That corresponds to a minimum, the three quartiles, and the maximum of the observations.
- We can plot the five-number summary using a boxplot or box-and-whisker plot.

Example

- Values are
  $14.7, 15.1, 14.5, 15.9, 14.6, 14.5, 14.4, 14.4, 10.2, 14.7, 16.4, 14.9, 14.1, 14.4, 14.7$
- Order Statistics
  $10.2, 14.1, 14.4, 14.4, 14.4, 14.5, 14.5, 14.6, 14.7, 14.7, 14.7, 14.9, 15.1, 15.9, 16.4$

$$n = 15$$

Q1 = (n+1)/4

Q2 = (n+1)/2

Q3 = 3(n+1)/4

$$Q_1 = \frac{15+1}{4} = 4$$

$$Q_2 = \frac{15+1}{2} = 8$$

$$Q_3 = \frac{3}{4} \times (15+1) = 12$$

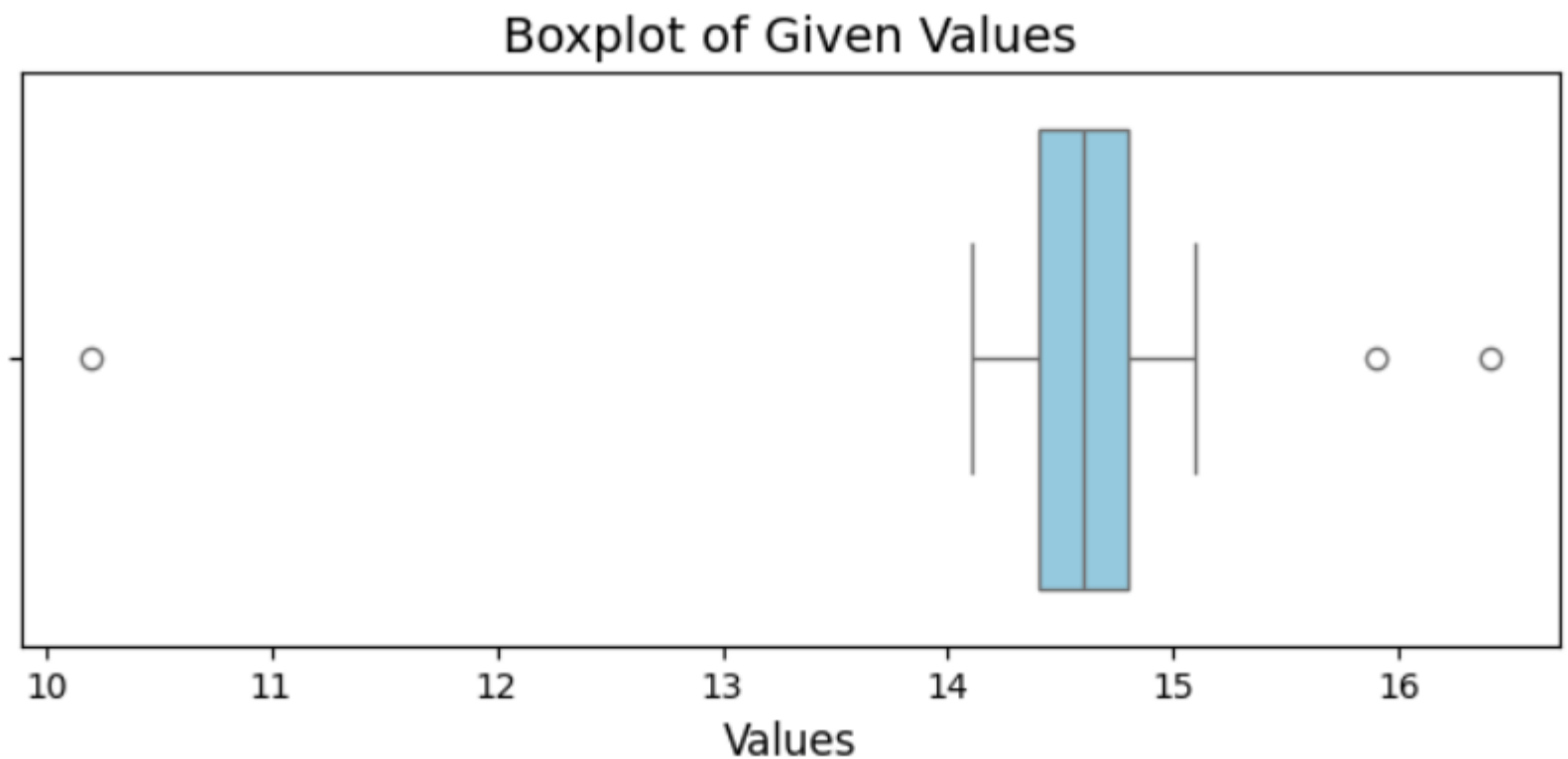
Q_1, Q_2, Q_3 are $4^{th}, 8^{th}, and\ 12^{th}$ values respectively. The values are $14.4, 14.6, 14.9$

Example

- IQR

$$14.9 - 14.4 = 0.5$$

- Outliers would be point below $Q_1 - 1.5 \times IQR$ and point above $Q_3 + 1.5 \times IQR$
- Outliers would be below $14.4 - 1.5 \times 0.5 = 14.4 - 0.75 = 13.65$
- And above $14.9 + 1.5 \times 0.5 = 14.9 + 0.75 = 15.65$
- So outliers are $10.2, 15.9, 16.4$

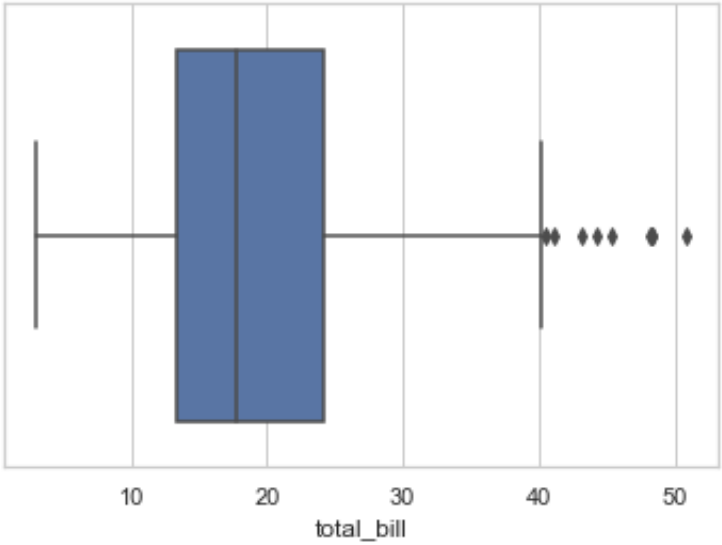


Samatrix.io

Box Plots-4

- A box plot has three graphical sections: box, whiskers, and outliers.
- 50% of the central data is contained in the box.
- The length of the box shows the interquartile range.
- The vertical line in the box shows the median.
- The length of the whiskers is 1.5 times the length of the box.
- It shows the range of data that is outside the box.
- The outliers are far from the center and they are denoted by dots or stars
- Box plot can show the symmetry as well as skewness of the box with respect to the median. Box plots can identify the outliers.

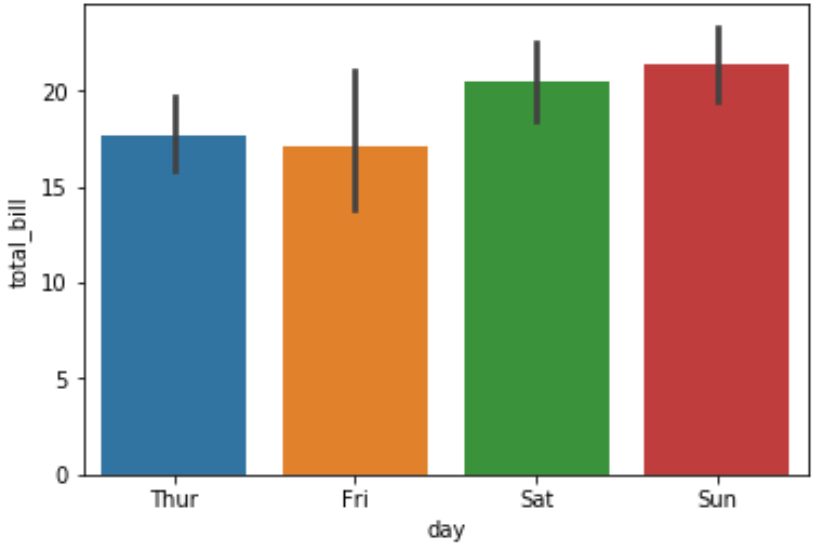
Box Plots using Python



- For python, we will use pandas and seaborn libraries to plot a boxplot
- The commands are

```
>>> import seaborn as sns
>>> sns.set(style="whitegrid")
>>> tips = sns.load_dataset("tips")
>>> ax = sns.boxplot(x=tips["total_bill"])
```

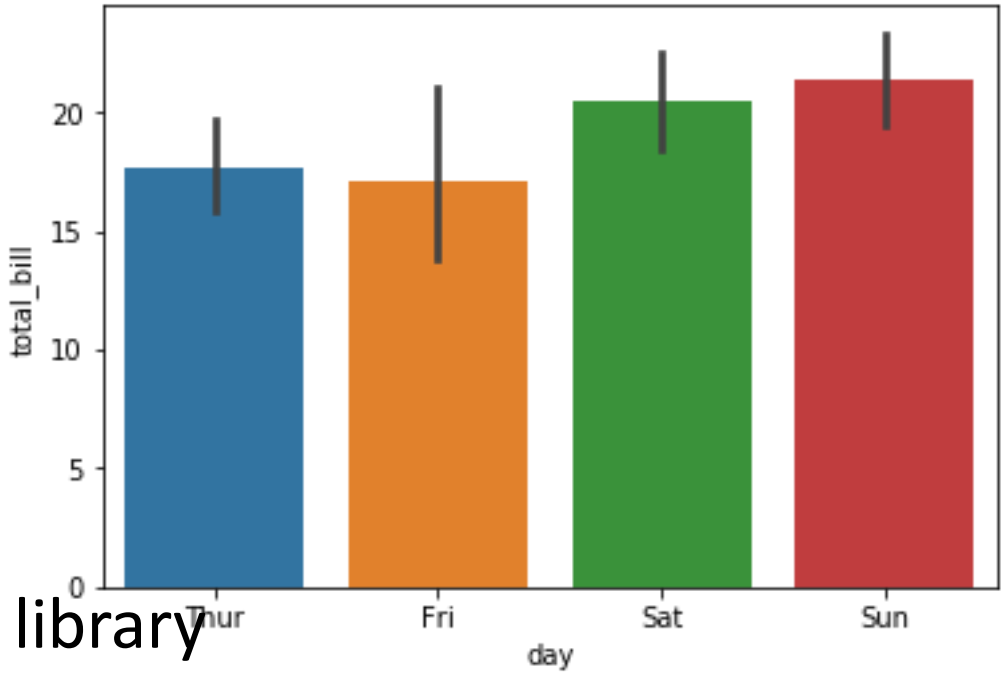
Bar Charts



- A bar chart is a graph (or chart), that is used to display the categorical data.
- It has a set of bars.
- One bar represents one category.
- From the height of the bar, we can find out the number of items in that category. It displays the important structure of the data such as which is the most common category, and which one is rare. We can plot then vertically as well as horizontally.

Days = [Mon,Tue,Wed,…]
Tips = [60,70,50,65,80]

Bar Charts



We can use seaborn library to plot the bar chart.

We have used the tips data set from seaborn library

```
>>> import seaborn as sns
>>> tips = sns.load_dataset("tips")
>>> ax = sns.barplot(x="day", y="total_bill", data=tips)
```

Frequency Tables

- Frequency table (or data binning) is another main approach to simplify a set of numbers. We follow the following steps
  1. First of all, partition the values into overlapping groups (bins). We can use equal width bins but that is not mandatory
  2. Secondly, place the items in each bin it belongs to
  3. Thirdly count the items in each bin

```
test_scores = [45, 67, 45, 56, 78, 89, 56, 45,
67, 90, 56, 76, 89, 90, 67, 56, 45, 78, 67, 89]

Create frequency table for this data
```

Frequency Tables – Trade off

- Even though the frequency table help summarize data into an understandable form, but we may lose some information in summary tables.
- There is a tradeoff between ease of understanding the information and loss of information.
- When we place a number in a bin, we lose some value able information in the summary.
- Every bin has a boundary.
- When the information lies within the boundary, its exact information is lost.
- Reduction in the number of bins will increase loss of information.
- Increase in number of bins will lead to reduction in information loss but our data summary will be less concise.

Frequency Tables – Discrete Variables

```
>>> import seaborn as sns
>>> tips = sns.load_dataset("tips")
>>> frequency, bins = np.histogram(tips["tip"], bins=9)
>>> for b, f in zip(bins[1:], frequency):
...     print(b,' ',f)

2.0   45
3.0   78
4.0   68
5.0   25
6.0   20
7.0   5
8.0   1
9.0   0
10.0   2
```

Frequency Tables –Categorical Data

For this we have used crosstab() function of pandas

```
>>> import pandas as pd
>>> tips = pd.read_csv("./Data/tips.csv")
>>> tip = pd.crosstab(index=tips["day"],columns="No of Customers")
>>> tip.index=["Friday","Saturday","Sunday","Thursday"]
>>> tip
col_0     No of Customers
Friday                19
Saturday               87
Sunday                76
Thursday               62
```

Frequency Tables –Categorical Data 2 Variable

We have used margins=True for adding marginal values

```
>>> import pandas as pd
>>> tips = pd.read_csv("./Data/tips.csv")
>>> day_time = pd.crosstab(index=tips["day"], columns=tips["time"], margins=True)
>>> day_time.index=["Friday","Saturday","Sunday", "Thursday","Time Total"]
>>> day_time.columns=["Dinner","Lunch","Day Total"]
>>> day_time
```

	Dinner	Lunch	Day Total
Friday	12	7	19
Saturday	87	0	87
Sunday	76	0	76
Thursday	1	61	62
Time Total	176	68	244

Samatrix.io

Frequency Tables –Categorical Data 2 Variable

The crosstab() function also helps us to get the total proportion of counts in each cell

```
>>> import pandas as pd
>>> tips = pd.read_csv("./Data/tips.csv")
>>> day_time = pd.crosstab(index=tips["day"], columns=tips["time"], margins=True)
>>> day_time.index=["Friday","Saturday","Sunday", "Thursday","Time Total"]
>>> day_time.columns=["Dinner","Lunch","Day Total"]
>>> day_time/day_time.loc["Time Total","Day Total"]
```

	Dinner	Lunch	Day Total
Friday	0.049180	0.028689	0.077869
Saturday	0.356557	0.000000	0.356557
Sunday	0.311475	0.000000	0.311475
Thursday	0.004098	0.250000	0.254098
Time Total	0.721311	0.278689	1.000000

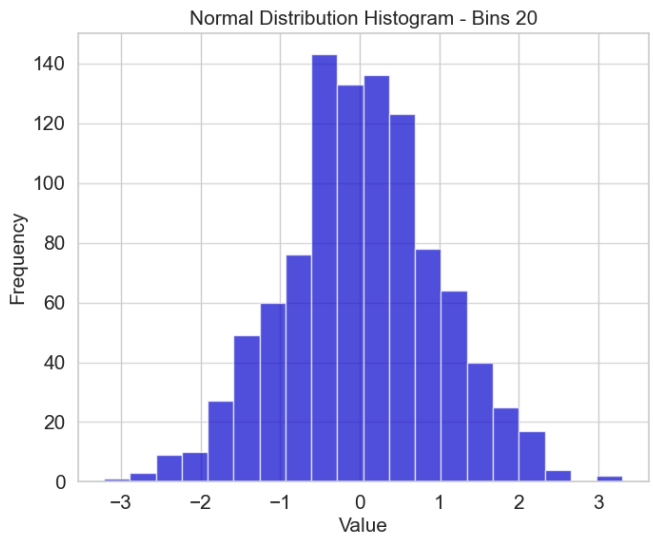
Histograms

- The histogram is a way to show the data distribution in the frequency table.
- To construct the histogram, we need to follow the following steps
  - The group boundaries should be on horizontal axis on a linear scale
  - Draw rectangular bars corresponding to each group. The area of the bars should be proportional to the frequency of the respective group.
  - There should not be any gaps between the bars for continuous data
  - The scale on the vertical axis represents density. Density is the frequency of the group divided by group width. If the width of the groups is equal, the scale is proportional to frequency or relative frequency and we can use frequency instead of density. The labeling on the vertical axis of the graph is not necessary because the shape of the graph is important not the vertical axis

Histograms – Tradeoffs

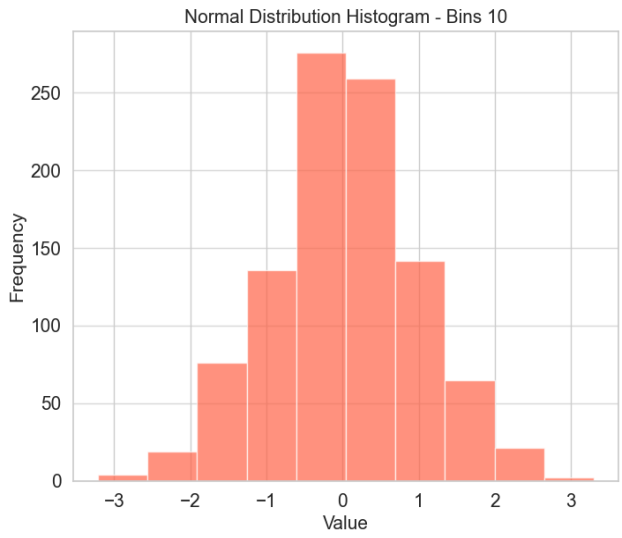
- From histogram, we get a picture about sample data distribution.
- We can understand the shape of the distribution.
- The distribution represents the underlying population from where the sample was drawn.
- The population distribution is smoother than the sample distribution.
- We need to decide the number of groups.
- If there are too many groups, the histogram shape would be of saw tooth appearance and the sample histogram would not be able to represent the population correctly.
- If number of groups are less, we may lose the details about the shape.
- So, there is a trade-off between too few groups and too many groups.

Histograms – Using Python



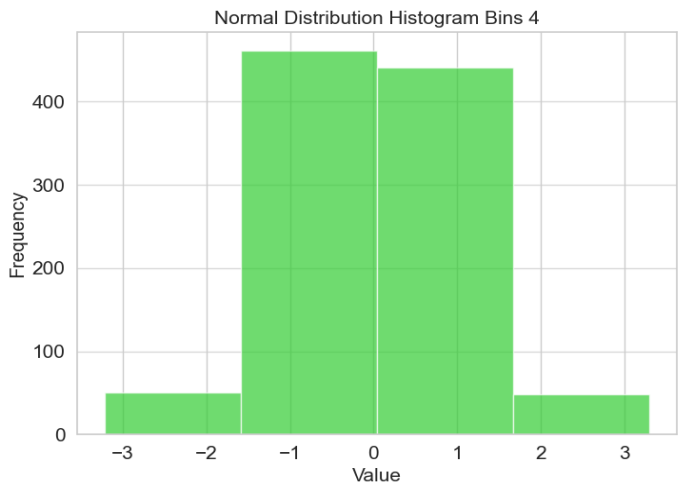

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> np_hist = np.random.normal(loc=0, scale=1, size=1000)
>>> plt.figure(figsize=[10,8])
>>> n, bins, patches = plt.hist(x=np_hist, bins=20, color='#0604cc',alpha=0.7)
>>> plt.grid(axis='y', alpha=0.75)
>>> plt.xlabel('Value',fontsize=15)
>>> plt.ylabel('Frequency',fontsize=15)
>>> plt.xticks(fontsize=15)
>>> plt.yticks(fontsize=15)
>>> plt.title('Normal Distribution Histogram - Bins 20',fontsize=15)
>>> plt.show()
```

Histograms – Using Python Bin 10




```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> np_hist = np.random.normal(loc=0, scale=1, size=1000)
>>> plt.figure(figsize=[10,8])
>>> n, bins, patches = plt.hist(x=np_hist, bins=10, color='#FF6347',alpha=0.7)
>>> plt.grid(axis='y', alpha=0.75)
>>> plt.xlabel('Value',fontsize=15)
>>> plt.ylabel('Frequency',fontsize=15)
>>> plt.xticks(fontsize=15)
>>> plt.yticks(fontsize=15)
>>> plt.title('Normal Distribution Histogram - Bins 10',fontsize=15)
>>> plt.show()
```

Histograms – Using Python Bin 4



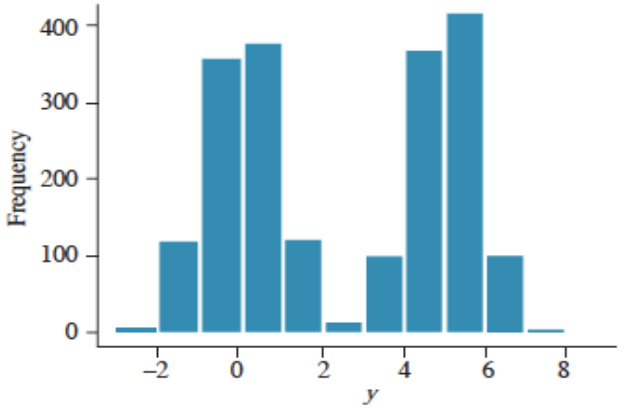

```
>>> import matplotlib.pyplot as plt
>>> import numpy as np
>>> np_hist = np.random.normal(loc=0, scale=1, size=1000)
>>> plt.figure(figsize=[10,8])
>>> n, bins, patches = plt.hist(x=np_hist, bins=4, color='#32CD32,alpha=0.7)
>>> plt.grid(axis='y', alpha=0.75)
>>> plt.xlabel('Value',fontsize=15)
>>> plt.ylabel('Frequency',fontsize=15)
>>> plt.xticks(fontsize=15)
>>> plt.yticks(fontsize=15)
>>> plt.title('Normal Distribution Histogram - Bins 4',fontsize=15)
>>> plt.show()
```

Histograms – Using Python Bin 4

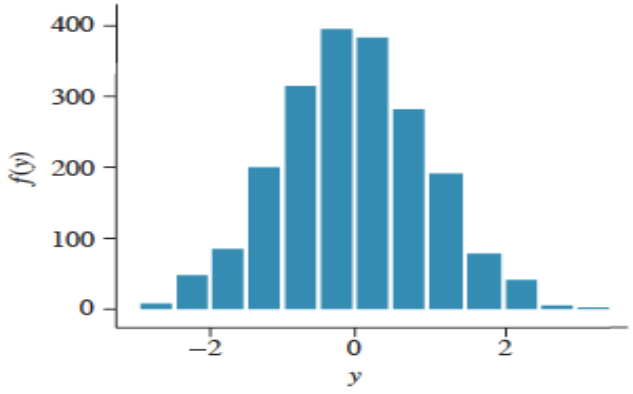
- The figure above shows the histogram of the sample data that is drawn from Normal (Gaussian) distribution.
- Sample size is 1000. We draw the histogram of the data using 20, 10, and 4 groups respectively.
- This is an illustration of trade-off between too many and too few bins.
- The charts above shows that the histogram with 20 bins has a saw-tooth appearance.
- The chart with 10 bins gives a better representation of a normal distribution.
- The histogram chart with 4 groups has lost too much details.

Common Shapes of Histogram

- **Unimodal Histogram** has one major peak whereas **Bimodal Histogram** has two major peaks.



Unimodal Histogram

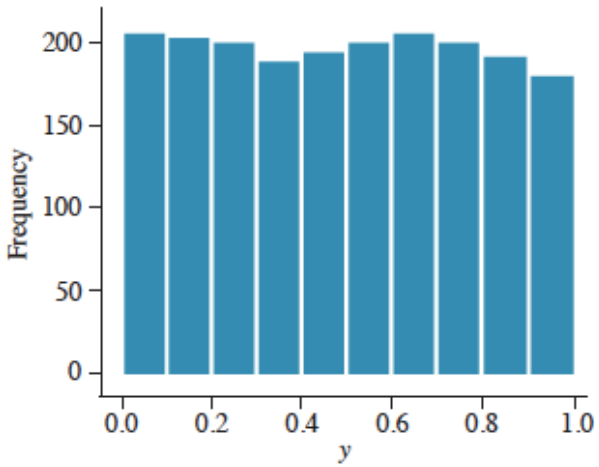


Unimodal Histogram/Symmetrical Histogram

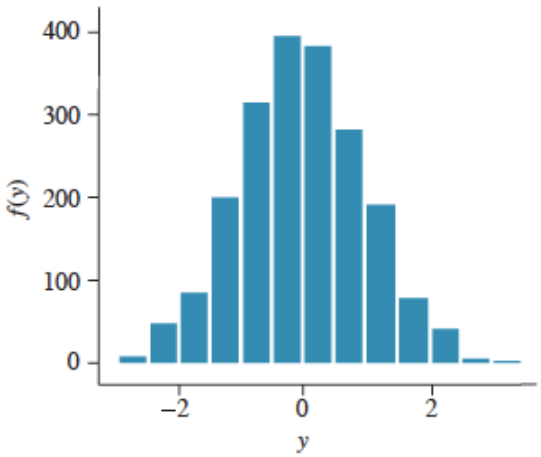
Samatrix.io

Uniform and Symmetric Histogram

- **Uniform Histogram** when every interval has same number of observations. When the left and right side of the histogram has same shape, the **Symmetric Histogram** is formed



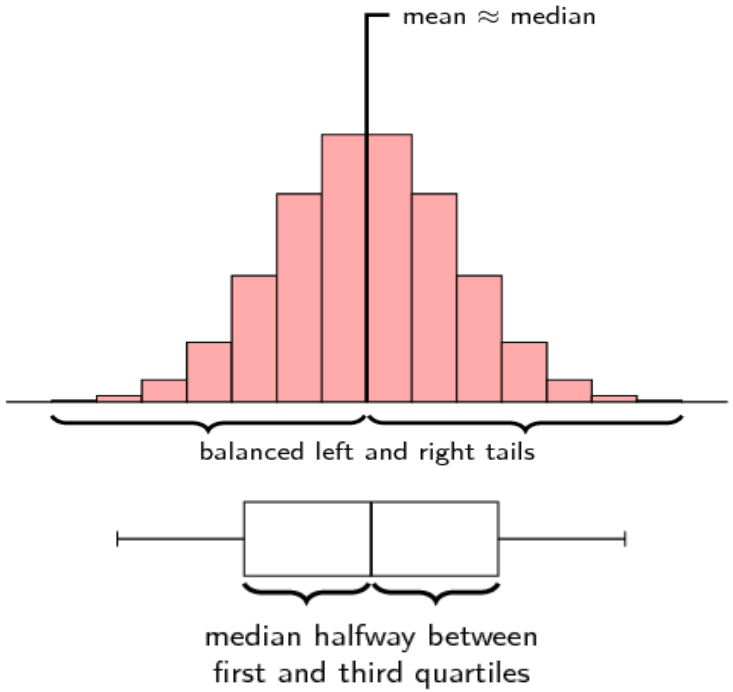
Uniform Histogram

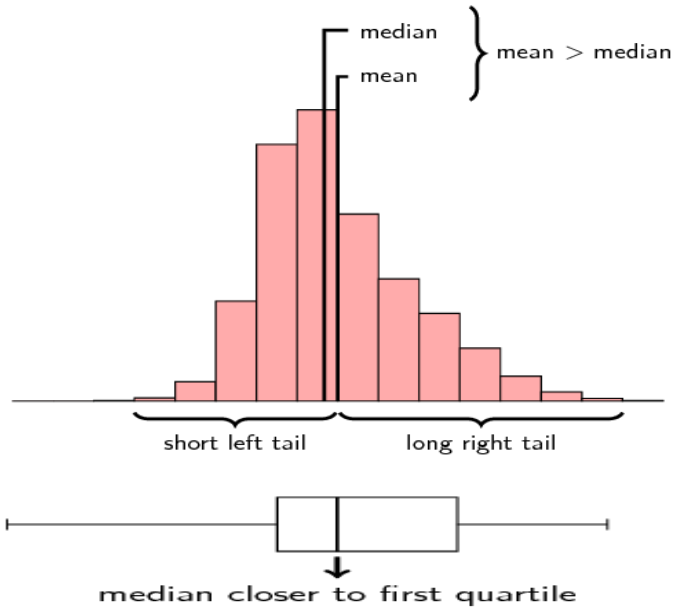


Unimodal Histogram/Symmetrical Histogram

Symmetric vs Skewed Histogram

- In symmetric histogram, both the halves are mirror image of each other. Mean and median are approximately equal. Median Q_2 falls in the middle of Q_1 and Q_3
- In case of **skewed histogram**, both the halves are not mirror image of each other. The tail of one side is longer than the tail of another side

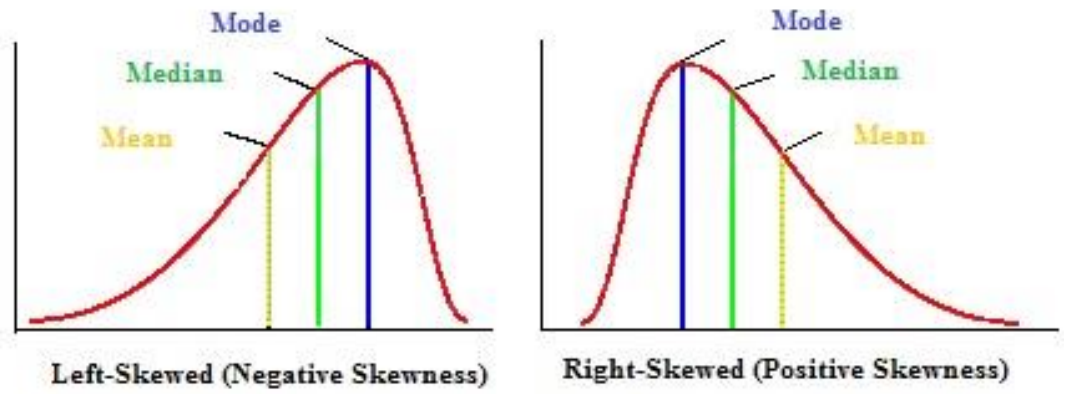




Samatrix.io

Right vs Left Skewed Histogram

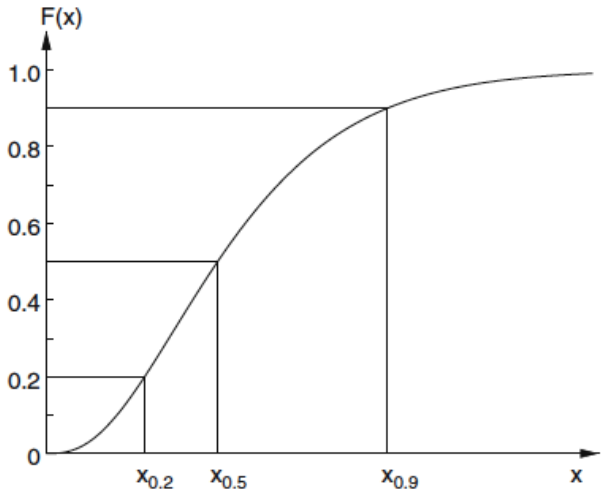
- If the longer tail is on the right side, the histogram is **skewed right**. If the longer tail is on the left side, the histogram is **skewed left**
- In the right skewed data, mean is generally larger than median whereas in the left skewed data, mean is generally smaller than the median



Cumulative Frequency Polygon

- Cumulative frequency polygon (also called **ogive**), is another way to display the data from a frequency table.
- The graph helps to estimate the median and quartiles. We should follow the following steps
- Using the cumulative frequency polygon graph, we can easily estimate the median and quartiles of the values in a group are equally distributed across the group.

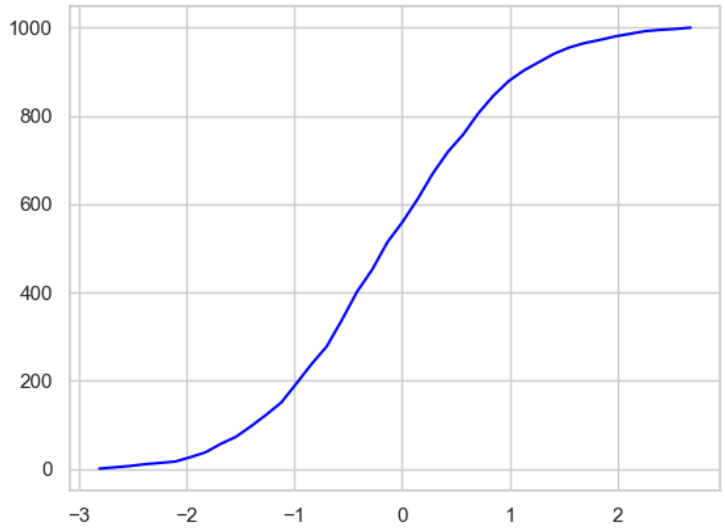
Cumulative Frequency Polygon



- First of all, you need to mark 50% on the vertical scale.
- From the 50% point, draw a horizontal line, that is parallel to the x-axis till it intersects the curve.
- From the point of intersection draw a vertical line till it intersects the x-axis.
- The point of intersection of the vertical line and the x-axis is the estimate of the median.
- Similarly, we can find the lower and upper quartiles respectively.

Samatrix.io

Cumulative Frequency Polygon




```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> data = np.random.randn(1000)
>>> values, base = np.histogram(data, bins=40)
>>> cumulative = np.cumsum(values)
>>> plt.plot(base[:-1], cumulative, c='blue')
>>> plt.show()
```

Measure of Central Tendency

Numerical Measures

- One of the important aspect of data analytics is to summarize the data set with numbers.
- Numerical descriptive measures help to convey appropriate picture of a phenomena verbally by creating a mental picture.
- Graphical descriptive measure cannot be helpful for statistical inference.
- A sample frequency histogram may not be similar its population frequency histogram.
- Measure of central tendency and measure of variability are two most important numerical descriptive measures.
- They tell us about the center of distribution of measurement and how they vary about the center.
- Numerical descriptive measures for a population are known as parameters whereas the numerical descriptive for a sample are known as statistics.
- For statistical inference problems, we can calculate the numerical descriptive measures for a sample and calculate the population parameters based on the quantities.

Example

Example: Suppose you are planning a trip to Goa. While preparing for the trip, you want to know about the weather so that you can pack your clothes accordingly. The previous year maximum temperature (In degree Celsius) for the corresponding month was:

$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

- From an individual value, we can get an understanding of the temperature in Goa, but it lacks summary.
- We can find an average of the 31 values by adding all the values and dividing by the number of observations.
- In this case average is $21 + 24 + 22 + \cdots + 28 + 29)/31 = 25.80$.
- From average we can understand what temperature we can expect in Goa.
- To decide on the right clothing, we need to know the variability in temperature.
- The variability is between 31 and 21. So we need three numbers to summarize the data $25.8, 31\ and\ 21$

Measure of central tendency

- Human tends to use “average” to make comparisons.
- Suppose you score 60% in an examination.
- In order to understand how well you did, you check the average score of your class.
- If average score of the class is 40%, you feel happy.
- If the average score of the class is 90%, you will not feel so happy.
- In our daily life, we tend to use several measures such as average temperature in January of a city, average income, average height etc.
- The statistical functions that we use to describe the average or center of the data as measure of central tendency
- The measure of central tendency tells us where **most of the data is located**

Measure of central tendency

- The most common measures of central tendency are the mean and the median.
- For this study, we have not considered mode (most common value) to be a suitable measure of central tendency.
- For the continuous data set each observation is generally unique.
- In many cases, we found the mode to be near one end of the distribution, not at the central region.
- The mode may not be unique for a given data set.

The mean

- The mean or arithmetic mean is a simple and effective summary of the data set. It is an intuitive measure of central tendency. For a sample of n values $\{x_i\}, i = 1, \dots, n$, mean

$$\mu\ (or\ \bar{x}) = \frac{1}{n}\sum_{i=1} x_i = \frac{1}{n} \times (x_1 + x_2 + \cdots + x_n)$$

- **The mean is simple, and it is easy to calculate mean**. You just need to add the numbers and divide the sum by number of observations in the sample
- **The mathematical properties of mean**: The mean of the sum is also the sum of the mean. For example, if the total income of every person in a community has three components from three sources. u_i, v_i, w_i. The total income of each person is $y_i = u_i + v_i + w_i$. The average income of the community is

$$\bar{y} = \bar{u} + \bar{v} + \bar{w}$$

- Where as $\bar{u}, \bar{v}, \bar{w}$ are average income of community from each source. Average income can also be calculated as

$$\bar{y} = \frac{1}{n} \times (y_1 + y_2 + \cdots + y_n)$$

- Hence, however we try to calculate the mean income of the community, we always find the same answer.

Combined Mean

- Mean combines
- The combined mean is the weighted average of the means of two or more separate groups, where the weights are the size of each group.
- For example, if the data comes from two sources such as males and females.
- The overall mean is the weighted mean of the average of male dataset and the weighted mean of the average of female dataset where weights are the proportion of male and female in the overall data set.

$$\bar{x} = \frac{m \times x_1 + n \times x_2}{m + n}$$

$$\bar{x} = \frac{m}{m + n} \times x_1 + \frac{n}{m + n} \times x_2$$

- The two most important reasons behind using weighted mean are
  - Some values in dataset will have more variability than others. We can give the higher variable values less weight
  - During our sampling process, we may realize that one group was underrepresented. We can correct that by giving more weight to the underrepresented group

Mean as center of gravity

- Mean is a center of gravity of numbers.
- Mean can be represented as the balance point if we place equal weights at each of the data points on a weightless number line.
- In that case, the mean would be the balance point.
- The presence of an outlier can lead to a big disadvantage.
- A single observation that is much bigger or smaller than the rest of the observations can have a big effect on the overall mean.
- In such cases that involve skewed data, the mean would be problematic

Useful facts about mean

- If you scale the data, the mean will also scale

$$mean(k \times x_i) = k \times mean(x_i)$$

- If you translate the data, the mean will also translate

$$mean(x_i + c) = mean(x_i) + c$$

- The sum of signed deviation from mean is always zero

$$\sum_{i=1}^{n} (x_i - \bar{x}) = 0$$

- The sum of squared distances of data points from mean is minimum.

Mean using Python

- Python helps in data analysis and statistics. The statistics module of python comes with functions like mean(), median(), and mode(). The function mean(), can be used to calculate the mean or average of a data set
- Example: Let's calculate the mean temperature of Goa for a month where the data set of daily temperature is as follows

$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

```
>>> import statistics
>>>temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,
26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> x = statistics.mean(temperature)
>>> print("Mean Temperature is :", x)
Mean Temperature is : 25.806451612903224
```

Exercise - 2

- Example: Let’s calculate the mean of the following data set
  - Positive integer numbers
  - Negative integer numbers
  - Mixed range of numbers
  - Fractional numbers
  - Dictionary of set of numbers whereas the key is considered for mean

- Positive Integer Numbers

```
>>> from statistics import mean
>>> pos_int = (12, 5, 3, 8, 6, 10, 11)
>>> print("Mean of data set 1 is % s" % (mean(pos_int)))
Mean of data set 1 is 7.857142857142857
```

- Negative Integer Numbers

```
>>> from statistics import mean
>>> neg_int = (-3, -1, -8, -11, -15, -9)
>>> print("Mean of data set 2 is % s" % (mean(neg_int)))
Mean of data set 2 is -7.833333333333333
```

Exercise - 2

- Mixed Integer Numbers

```
>>> from statistics import mean
>>> mix_int = (-3, -8, -16, 12, 9, 16, 8)
>>> print("Mean of data set 3 is % s" % (mean(mix_int)))
Mean of data set 3 is 2.5714285714285716
```

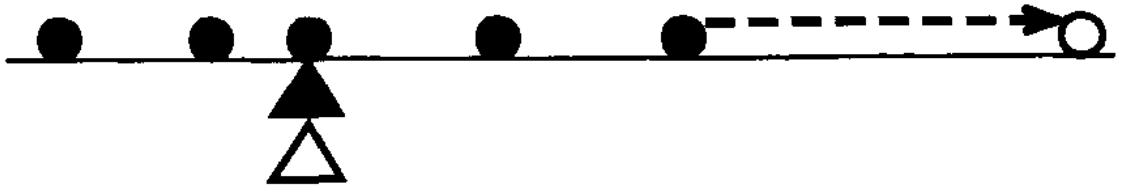
- Fractional Numbers (By using Fraction module)

```
>>> from statistics import mean
>>> from fractions import Fraction as fr
>>> frac_avg = (fr(2, 3), fr(66, 12), fr(20, 3), fr(20, 30))
>>> print("Mean of data set 4 is % s" % (mean(frac_avg)))
Mean of data set 4 is 27/8
```

- Dictionary of set of numbers

```
>>> from statistics import mean
>>> dict_avg = {1:"one", 2:"two", 3:"three"}
>>> print("Mean of data set 5 is % s" % (mean(dict_avg)))
Mean of data set 5 is 2
```

Median



- The median of a data set is the middle value when the data is arranged from the lowest to the largest.
- For a median value, 50% of the measurements is less than or equal to it, whereas 50% of the measurements is greater than or equal to it.
- When the sample size is odd, the median is the middle number. For the even-sized sample, median is the average of two closest numbers to the middle.
- The outliers do not influence the median at all due to which median is a suitable measure for highly skewed data.
- Median does not have as good mathematical properties and combining properties as mean
- The median of the sum may not be the same as the sum of medians.
- The median of combined data set may not be the weighted average of the medians
- Hence the median is not used very often as compared to the mean. However, for very skewed data such as income data, the outliers may influence the mean, but they cannot influence the median.

Median using Python

- The function median(), can be used to calculate the mean or average of a unsorted data set. You need not sort the data set before sending data as parameter to the median() function.
- Example: Let's calculate the median temperature of Goa for a month where the data set of daily temperature is as follows

$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

```
>>> import statistics
>>> temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> x = statistics.median(temperature)
>>> print("Median Temperature is :", x)
Median Temperature is : 26
```

Exercise - 2

- Example: Let’s calculate the median of the following data set
  - Positive integer numbers
  - Negative integer numbers
  - Mixed range of numbers
  - Fractional numbers
  - Dictionary of set of numbers whereas the key is considered for mean

- Positive Integer Numbers

```
>>> from statistics import median
>>> pos_int = (12, 5, 3, 8, 6, 10, 11)
>>> print("Median of data set 1 is % s" % (median(pos_int)))
Median of data set 1 is 8
```

- Negative Integer Numbers

```
>>> from statistics import median
>>> neg_int = (-3, -1, -8, -11, -15, -9)
>>> print("Median of data set 2 is % s" % (median(neg_int)))
Median of data set 2 is -8.5
```

Exercise - 2

- Mixed Integer Numbers

```
>>> from statistics import median
>>> mix_int = (-3, -8, -16, 12, 9, 16, 8)
>>> print("Median of data set 3 is % s" % (median(mix_int)))
Median of data set 3 is 8
```

- Fractional Numbers (By using Fraction module)

```
>>> from statistics import median
>>> from fractions import Fraction as fr
>>> frac_median = (fr(2, 3), fr(66, 12), fr(20, 3), fr(20, 30))
>>> print("Median of data set 4 is % s" % (median(frac_median)))
Median of data set 4 is 37/12
```

- Dictionary of set of numbers

```
>>> from statistics import median
>>> dict_median = {1:"one", 2:"two", 3:"three"}
>>> print("Median of data set 5 is % s" % (median(dict_median)))
Median of data set 5 is 2
```

Trimmed Mean

- Trimmed mean (or Adjusted Mean) is a method of calculating the mean in which first the dataset is ordered and then trimming a small designated percentage of lower and upper order statistics and then taking the average of remaining observations.
- The trimmed mean is used to eliminate the influence of outliers or observations that affect the traditional mean unfairly. It is very helpful for data set with extremely skewed distribution.
- A trimmed mean is a mean that is trimmed by $k\%$, where k is the sum of the percentage of values that we remove from both the upper and lower bounds.
- The trimming points follow the rule of thumb and hence are often an arbitrary method of setting the threshold.

Trimmed Mean using Python

- We will use the trim_mean() function of stats that is part of scipy module. We have used the threshold of 10% and 20% to calculate the same.
- Example: Let's calculate the trimmed mean temperature of Goa for a month where the data set of daily temperature is as follows

$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

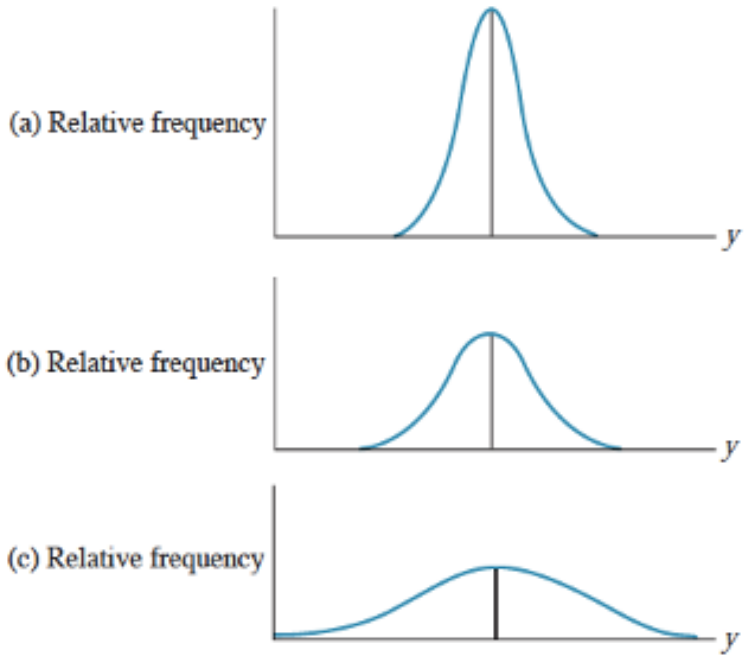
```
>>> from scipy import stats
>>> temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> x=stats.trim_mean(temperature, 0.1)
>>> print("Trimmed Mean Temperature at 10% Threshold :", x)
Trimmed Mean Temperature at 10% Threshold : 25.8
>>> x=stats.trim_mean(temperature, 0.2)
>>> print("Trimmed Mean Temperature at 20% Threshold :", x)
Trimmed Mean Temperature at 20% Threshold : 25.789473684210527
```

Measure of Variability

Measure of Spread

- Describing the data using measures of central tendency (mean or median) is not enough.
- We also need to determine how spread out is the resulting data distribution.
- In our statistics class, 50% of students may score 40 marks whereas remaining 50% of the students may score 60 marks.
- In another section, first, 50% of the students may score 30 marks whereas remaining 50% of the students may score 70 marks.
- The mean score in both the cases is 50 marks but the data is more spread in the second case than the first case.
- In the above example, the data in the second case is more spread out so data is more variable. Measuring the spread of the data set gives us the variability.

Measure of Spread



Data in case (c) is more variable than in case (b) and (a). Data in case (b) is more variable than in case (a) even though the mean is same in all the three cases

Samatrix.io

Range

- The difference between the largest and the smallest value of the observation is called Range.
- Calculating the range is very easy.
- But the largest and the smallest observations may be the outliers.
- The range is heavily influenced by outliers.

Range using Python

- To calculate range, we can use max()and min() functions from standard python library
- Example: Let's calculate the range of temperature of Goa for a month where the data set of daily temperature is as follows

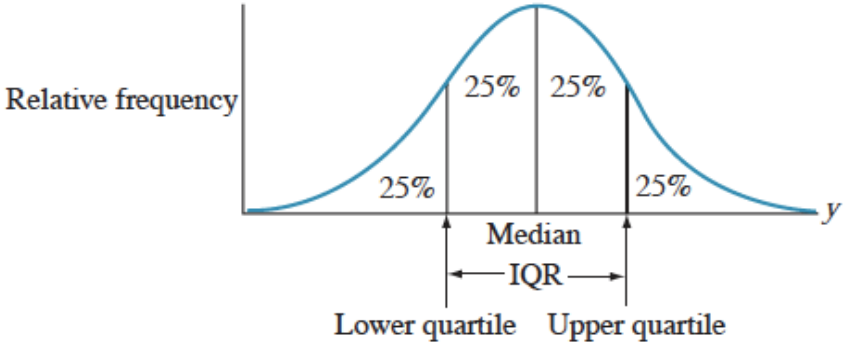
$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

```
>>>temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Temperature Range is :", max(temperature) - min(temperature))
Temperature Range is : 10
```

- We can use also np.max() and np.min() functions from NumPy

```
>>> import numpy as np
>>> temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Temperature Range is :", np.max(temperature) - np.min(temperature))
Temperature Range is : 10
```

Interquartile Range



- The interquartile range (IQR) measures the spread of middle 50% (between 75th and 25th percentile) of the observations.

$$IQR = Q_3 - Q_1$$

- The outliers do not influence the interquartile range. Even though IQR is measure of spread similar to variance and standard deviation, but it is not used very frequently in inference problems because of not having good math and combining properties

IQR using Python

- For Interquartile Range, we will use iqr() function of stats module of scipy
- Example: Let’s calculate the iqr of temperature of Goa for a month where the data set of daily temperature is as follows

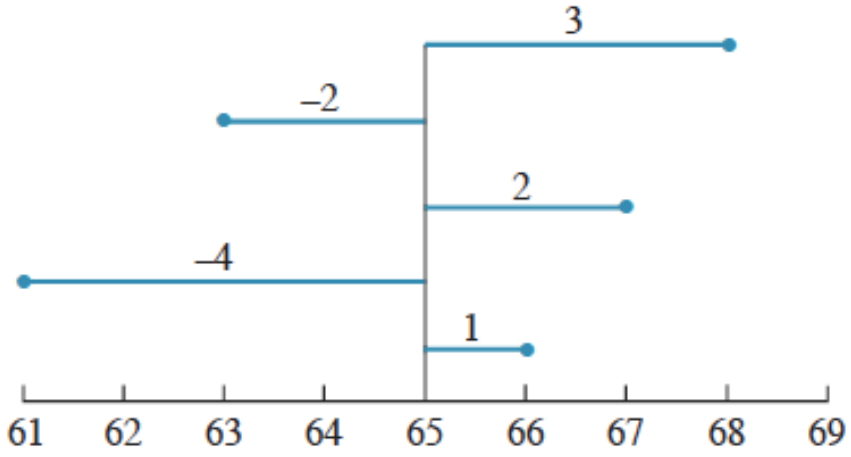
$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29,$$

```
>>> from scipy.stats import iqr
>>> temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Temperature IQR is :", iqr(temperature))
Temperature IQR is : 4.0
```

Deviation

- In statistics, we can measure the difference between the observed value and variable's mean.
- The measure is called Deviation.
- Deviation has a magnitude and a sign.
- The magnitude of the deviation represents the size of difference between both the values.
- The sign of deviation provides the direction of the difference. The positive deviation means that the observed value is higher than the refence value.
- The deviation is often referred as **residual** and **error**

Example



- Example: Suppose the sample measurement represents the weight of 5 students in a class. The measurement of the weight is $x_1 = 68, x2 = 67, x_3 = 66, x_4 = 63, x_5 = 61$. We have plotted the measurement on the dot diagram as shown below.
- The sample mean is

$$\bar{x} = \frac{1}{n} \times \sum x_i = \frac{325}{5} = 65$$

- We have plotted the sample mean on the dot diagram. The deviation of the measurements can be shown by $(x_i - \bar{x})$. The five measurements and their deviation from the mean is given below

Samatrix.io

Deviation

- The data set with low variability will have most of the measurement located near the mean. The dataset with high variability will disperse more around the mean.
- Even though the mean deviation cannot be used as the measure of variability, because it will always be equal to zero, many other measures of variability use deviation $(x_i - \bar{x})$

Variance

- Mean Square Deviation (MSD) or Mean Square Error (MSE) from mean is known as Variance.

$$\sigma^2 = \frac{1}{n} \times \sum_{i=1}^{n} (x_i - \bar{x})^2$$

- In this case, we have used the divisor n for defining variance. Some statisticians use $(n-1)$ as divisor. In this case, one degree of freedom is lost because we are using the sample mean instead of the population mean. When we use $(n-1)$ as a divisor, we refer to the sample estimate of variance instead of variance.
- Variance has good mathematical properties. They are more complicated than the mathematical properties of the mean. Variance also has good combining properties.
- The values that are far from the mean has a larger impact on variance because they already have a high deviation in either a positive or negative direction. If we square them, the deviation value becomes larger and all positive. Thus, outliers influence variance significantly.
- The unit of variance is squared so its size cannot be comparable to the mean

Variance using Python

- Example: Let's calculate the variance of temperature of Goa for a month where the data set of daily temperature is as follows

$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

- Let's compute the variance using var() function of NumPy as well as variance() and pvariance() functions of statistics module.
- The variance() function is for sample variance and uses $(n-1)$ in the denominator where as pvariance()is for population variance and uses n in the denominator.
- var() function of NumPy also represents population variance and uses n in the denominator.
- We can calculate the sample variance using NumPy by passing ddof=1 parameter to var() function. The default value of ddof=0

Variance using Python

```
>>> import statistics
>>> temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Sample Variance is :",statistics.variance(temperature))
Sample Variance is : 7.894623655913978
>>> print("Population Variance :",statistics.pvariance(temperature))
Population Variance is : 7.639958376690947
```

```
>>> import numpy as np
>>> temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Variance using Numpy is:",np.var(temperature))
Variance using Numpy is: 7.639958376690949
```

Standard Deviation

- Standard Deviation is the positive square root of the variance. It is also known as Root Mean Square (RMS).

$$\sigma = \sqrt{\frac{1}{n} \times \sum_{i=1}^{n} (x_i - \bar{x})^2}$$

- Standard Deviation is also affected by outliers even though not as much affected as the variance. It inherits good mathematical and combining properties from the variance.
- The most commonly used measure of spread is standard deviation. It has the same unit as mean so it is directly comparable to the mean

Standard Deviation using Python

- Example: Let's calculate the standard deviation of temperature of Goa for a month where the data set of daily temperature is as follows

$$21, 24, 22, 21, 23, 26, 24, 29, 28, 30, 31, 28, 24, 26, 27, 25, 26, 26, 25, 27, 26, 29, 30, 29, 29, 28, 24, 23, 22, 23, 24$$

- Let's compute the standard deviation using std() function of NumPy as well as stdev() and pstdev() functions of statistics module.
- The stdev() function is for sample variance and uses $(n-1)$ in the denominator where as pstdev()is for population variance and uses n in the denominator.
- std() function of NumPy also represents population variance and uses n in the denominator.
- We can calculate the sample variance using NumPy by passing ddof=1 parameter to std() function. The default value of ddof=0

Standard Deviation using Python

```
>>> import statistics
>>>temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Sample Standard Deviation is:", statistics.stdev(temperature))
Sample Standard Deviation is : 2.8097372930425326
>>> print("Population Standard Deviation is:", statistics.pstdev(temperature))
Population Standard Deviation is : 2.7640474628144407
```

Using Numpy

```
>>> import numpy as np
>>>temperature=[21,24,22,21,23,26,24,29,28,30,31,28,24,26,27,25,26,26,25,27,26,29,30,29,29,28,24,23,22,23,24]
>>> print("Population Std Dev using Numpy is:",np.std(temperature))
Population Std Dev using Numpy is: 2.764047462814441
>>> print("Sample Std Dev using Numpy is:",np.std(temperature, ddof=1))
Sample Std Dev using Numpy is: 2.809737293042533
```

Displaying relationship between two or more variables

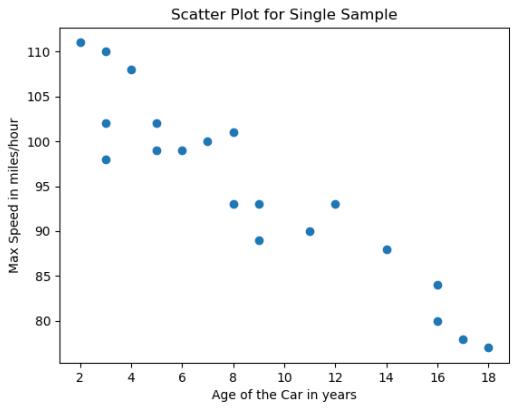
Bivariate Data

- In the previous section, we discussed the techniques of summarizing data from a single variable.
- But we need to study more than one variable at the same time.
- We may interest in studying individual variables separately as well as study the relations among the variable.
- When our dataset has measurements for two variables, we call it **bivariate data**, which is used to investigate the relationship between two variables.

Scatterplot

- The scatterplot is an ordinary 2-dimensional dotplot.
- It has the first variable on the horizontal axis (x-axis) and the second variable on the vertical axis (y-axis).
- We plot each point on the graph.
- By plotting all the points, we get the point cloud.
- The shape of the point cloud helps us determine whether there is a relationship between the two variables.
- If there is one, what type of relationship exists between them.
- For two samples of bivariate data, we can plot the points for both the samples on the same plot by using different symbols or colors for each sample and see if the two samples have similar relationships between the two variables.

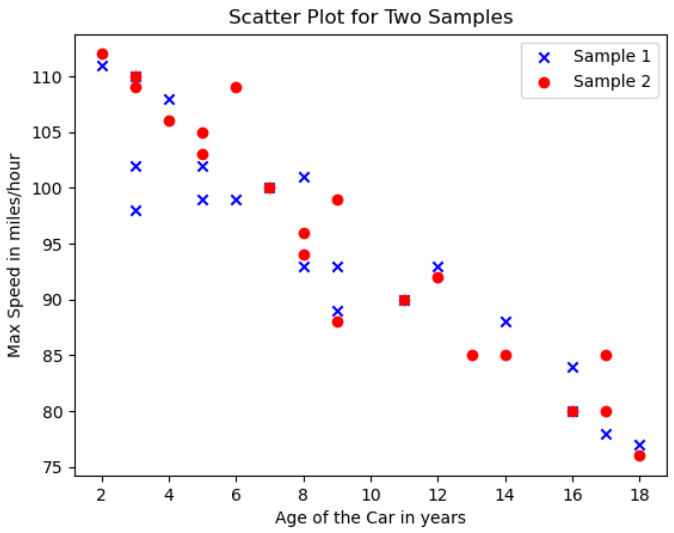
Scatterplot




```
>>> import matplotlib.pyplot as plt
>>> x = [3,14,6,3,16,17,9,12,4,9,5,8,16,18,8,3,11,7,2,5]
>>> y = [98,88,99,102,80,78,89,93,108,93,99,93,84,77,101,110,90,100,111,102]
>>> plt.scatter(x, y)
>>> plt.xlabel("Age of the Car in years")
>>> plt.ylabel("Max Speed in miles/hour")
>>> plt.title("Scatter Plot for Single Sample")
>>> plt.show()
```

- In this example, we have plotted the respective values of x and y. The x-axis represents the age of the car and the y-axis represents the speed of the car. From the data, it is evident that the faster car was 2 years old and the slowest car was 18 years old. With the scatter plot diagram, we can infer that with the increase in the age of the car, the speed reduces. So, there is an inverse relationship between the variables.

Scatterplot 2 Variables




```
>>> import matplotlib.pyplot as plt
>>> x = [3,14,6,3,16,17,9,12,4,9,5,8,16,18,8,3,11,7,2,5]
>>> y = [98,88,99,102,80,78,89,93,108,93,99,93,84,77,101,110,90,100,111,102]
>>> x1 = [11,7,2,5,17,9,12,13,14,6,3,17,8,3,4,9,5,8,16,18]
>>> y1 =[90,100,112,105,85,88,92,85,85,109,110,80,96,109,106,99,103,94,80,76]
>>> plt.scatter(x, y, c='b', marker='x', label='Sample 1')
>>> plt.scatter(x1, y1, c='r', marker='o', label='Sample 2')
>>> plt.xlabel("Age of the Car in years")
>>> plt.ylabel("Max Speed in miles/hour")
>>> plt.title("Scatter Plot for Two Samples")
>>> plt.legend(loc='upper right')
>>> plt.show()
```

- In this example, we have plotted a scatter plot diagram of two samples on a single plot. We can infer that both the samples demonstrate a similar relationship between the two variables.

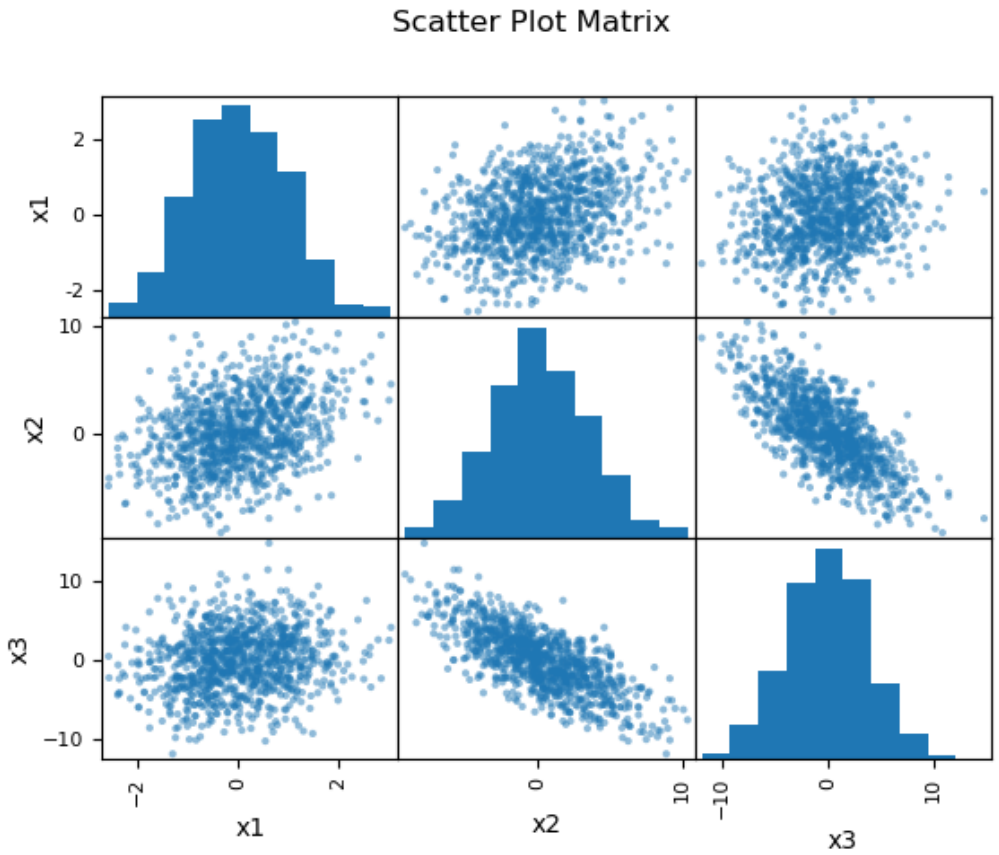
Samatrix.io

Scatterplot Matrix

- For many experiments, our dataset consists of measurements of multiple variables.
- Such a dataset is called **multivariate** data.
- We use a scatterplot matrix to study the relationship between variables.
- We plot the scatterplot matrix for each pair of variables.
- We show them in a matrix form.

Scatterplot Matrix

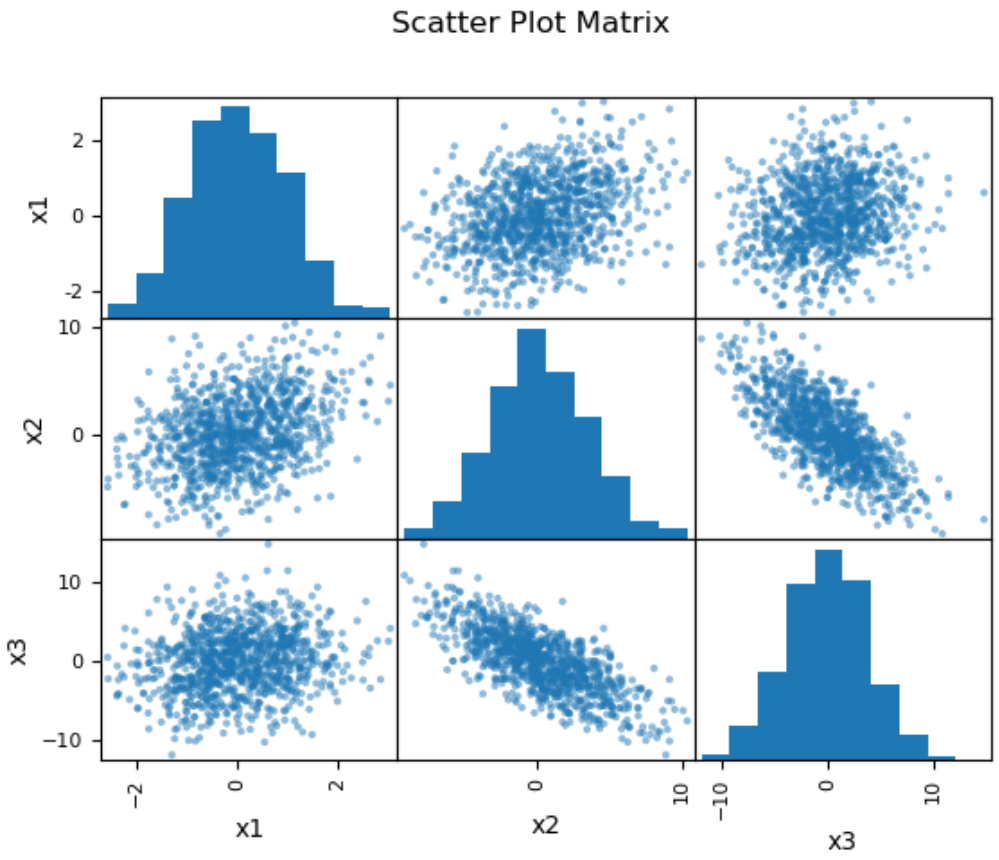
```
>>> import numpy as np
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>> np.random.seed(100)
>>> N = 1000
>>> x1 = np.random.normal(0, 1, N)
>>> x2 = x1 + np.random.normal(0, 3, N)
>>> x3 = 2 * x1 - x2 + np.random.normal(0, 2, N)
>>> df.head()
        x1        x2        x3
0 -0.224315 -8.840152  10.145993
1  1.337257  2.383882  -1.854636
2  0.882366  3.544989  -1.117054
3  0.295153 -3.844863   3.634823
4  0.780587 -0.465342   2.121288
>>> pd.plotting.scatter_matrix(df)
>>> plt.suptitle("Scatter Plot Matrix")
>>> plt.show()
```



Samatrix.io

Scatterplot Matrix

- At the diagonal, we the plot shows the distribution of the three measurements of our dataset.
- In other cells, there is a scatterplot between two variables at a time.
- Second column of the first row shows the scatterplot between x1 and x2
- Third column of the first row shows the scatterplot between x1 and x3
- First column of the second row shows the scatterplot between x1 and x2
- Third column of the second row shows the scatterplot between x2 and x3
- First column of the third row shows the scatterplot between x1 and x3
- Second column of the third row shows the scatterplot between x2 and x3

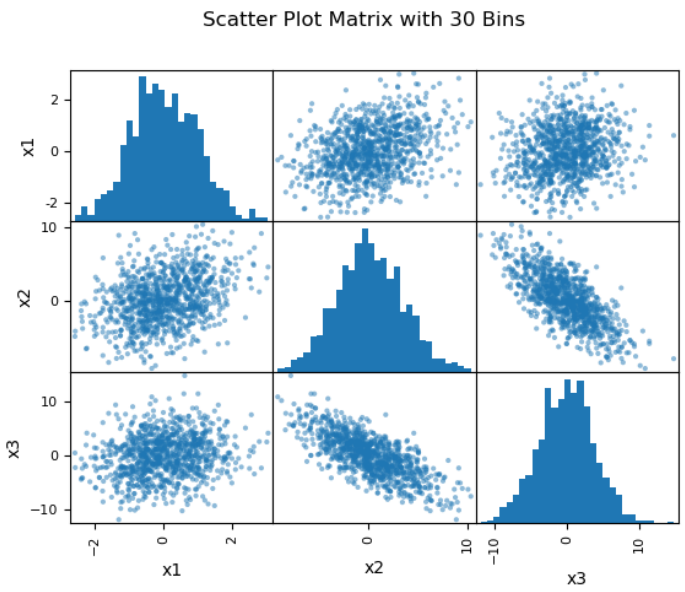


Samatrix.io

Scatterplot Matrix

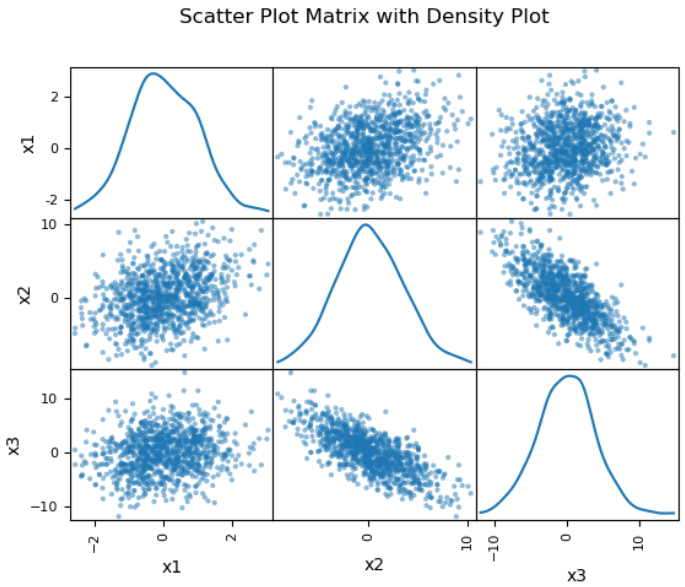
- We can change the number of bins by adding

```
>>> pd.plotting.scatter_matrix(df, hist_kwds={'bins':30})
```



- We can change to density plot instead of the histogram at the diagonal

```
>>> pd.plotting.scatter_matrix(df,
diagonal='kde')
```



Measure of association for two or more variables

Covariance

- As we have seen that we use variance to measure the variability of a variable. The **covariance** is used to measure the covariation or association between two variables.
- The covariance of two variables is the average of the difference between first variable and its mean times difference between second variable minus its mean
- $Cov(x, y) = \frac{1}{n} \times \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$
- This show the manner in which the variables vary together.

Height = 150,155,158,160,165

Weight = 55,59,60,61,65

No. of working hours = 8,6,7,4,5

Generate correlation Matrix for this data

1. Mean
2. Standard deviation
3. Variance
4. Covariance
5. Correlation

Correlation

- We can define the **correlation** between two variables as the covariance of the two variables divided by the product of standard deviation of the two variables. The value of correlation lies between -1 and +1

$$Corr(x, y) = r = \frac{Cov(x, y)}{\sqrt{Var(x) \times Var(y)}} = \frac{Cov(x, y)}{\sigma_{xx} \times \sigma_{yy}}$$

- With the correlation, we can measure the strength of the linear relationship between two variables
- A correlation of +1 shows that the points are on a straight line with positive slope
- A correlation of -1 shows that the points are on a straight line with negative slope
- A correlation of 0 shows that there is no relationship

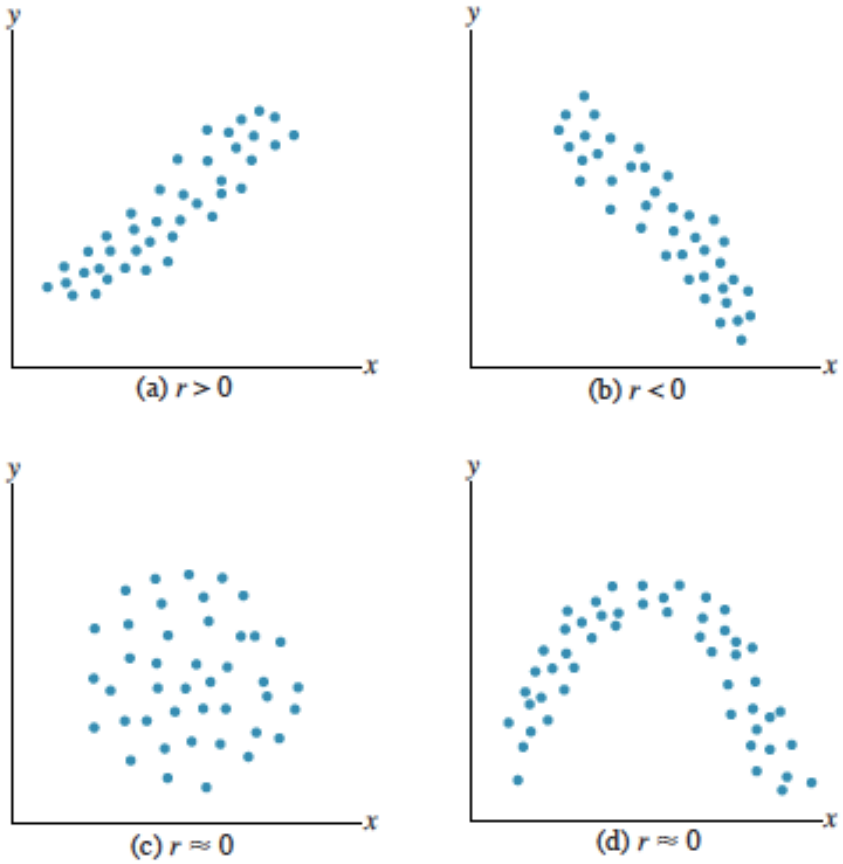
Correlation

Fig (a) – a strong positive relationship between both the variables

Fig (b) – a strong negative relationship between both the variables

Fig(c) – No relationship between both variables

Fig (d) – Strong relationship between both the variables but $r = 0$. This is due to the symmetric positive and negative relationship canceling each other.



Exercise Pyhton

- Example: An automobile parts manufacturing company is trying to establish a relationship between the hours spent on recreation activities for employees and no of items produced.

- For this study, we will create the random data for our analysis. We collected the data for 200 days.

Exercise Python

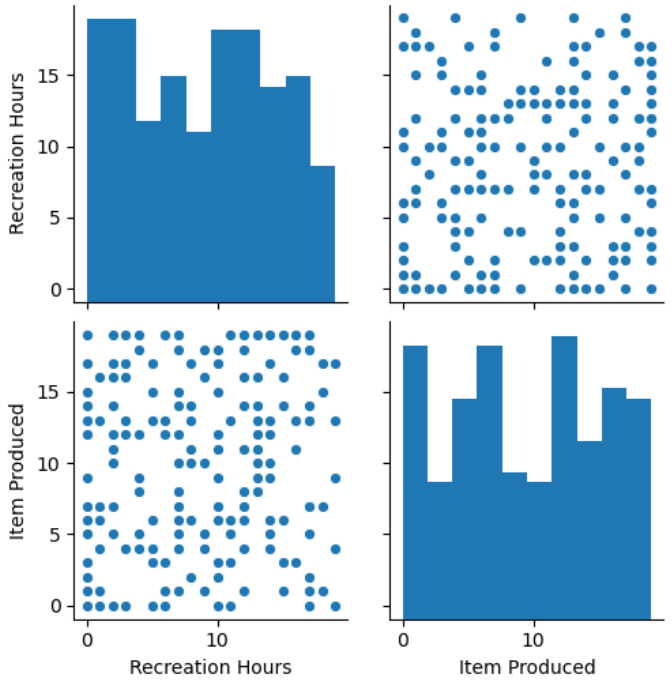
```
>>> import pandas as pd
>>> import numpy as np
>>> import seaborn as sns
>>> np.random.seed(2500)
>>> df = pd.DataFrame(np.random.randint(low= 0, high= 20, size= (200, 2)),
...                   columns= ['Recreation Hours', 'Item Produced'])
>>> df
    Recreation Hours  Item Produced
0                  4             19
1                  0              0
2                  1              6
3                 11             19
4                  2             17
..               ...            ...
195                7              5
196                6             19
197                3             17
198               13             19
199                1              1

[200 rows x 2 columns]
```

Samatrix.io

Exercise Python

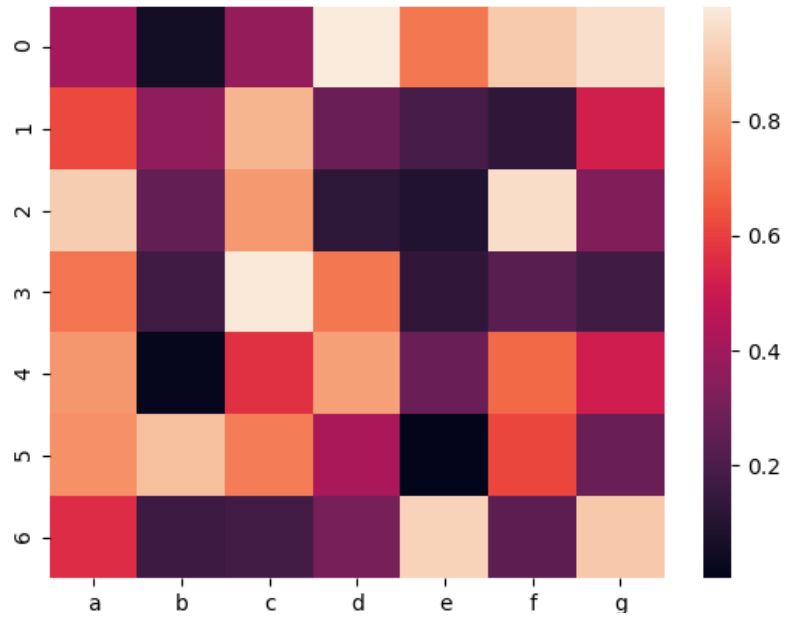
```
>>> df.agg(["mean","std"])
     Recreation Hours  Item Produced
mean          8.890000       9.515000
std           5.764028       5.949518
>>> df.var()
Recreation Hours    33.224020
Item Produced       35.396759
dtype: float64
>>> df.cov()
                Recreation Hours  Item Produced
Recreation Hours        33.224020       2.489095
Item Produced            2.489095      35.396759
>>> df.corr()
                Recreation Hours  Item Produced
Recreation Hours         1.000000       0.072583
Item Produced            0.072583       1.000000
>>> sns.pairplot(df)
```

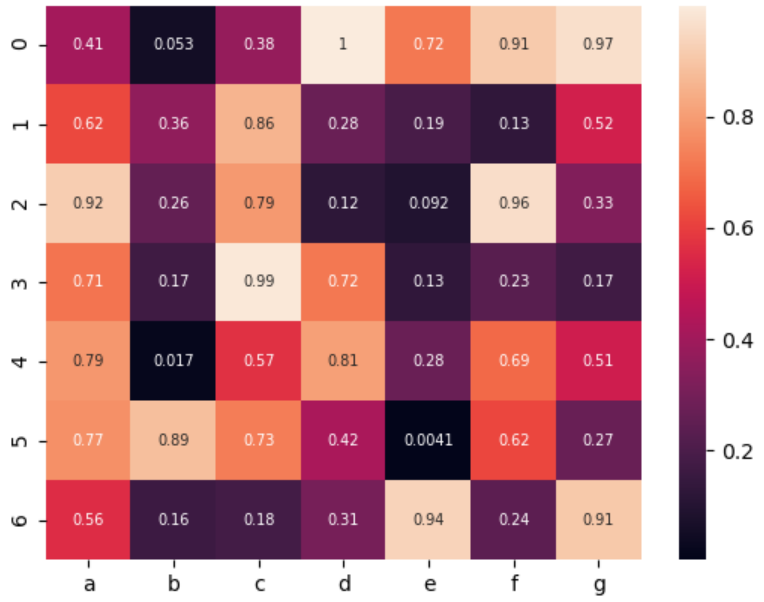


Samatrix.io

Heatmap

```
>>> import numpy as np
>>> import pandas as pd
>>> import matplotlib.pyplot as plt
>>> import seaborn as sns
>>> df=pd.DataFrame(np.random.random((7,7)),columns=['a','b','c','d','e','f','g'])
>>> sns.heatmap(df)
>>> sn.heatmap(df,annot=True,annot_kws={'size': 7}
```





Samatrix.io

Thanks

Samatrix Consulting Pvt Ltd

Samatrix.io